SOFTWARE ENGINEERING

Lab 7

Fuzzing for Input Validation Bugs

Property-based fuzz testing of a shipping quotation processor with the Hypothesis library

Name	Siddhesh Dattatray Koditkar
SRN	PES2UG24CS502
Course	Software Engineering
Lab	Lab 7 — Fuzzing for Input Validation Bugs
Repository	L6_L7_04_shipping_quotation · Lab 7
Date	13 November 2026

Deliverables: this report (PDF), the hardened processor.py and the completed test_processor.py.

1. Objective

The goal of this lab is to use fuzz testing as a security and quality-assurance technique. A deliberately fragile Python module (processor.py) implements three shipping-quotation helpers. Hypothesis generates hundreds of random and malformed inputs for those helpers, the resulting crashes are analysed, and the module is hardened so that malformed input is rejected cleanly instead of producing an unhandled traceback.

Learning outcomes demonstrated: applying property-based fuzzing with Hypothesis, distinguishing a crash from a deliberate rejection, analysing edge-case failures, and hardening code through defensive programming.

2. Step 1 — Environment and dependencies

The lab requires Python 3.8+ together with hypothesis (the fuzzing engine) and pytest (the test runner). Both packages were installed into the working environment and their versions were recorded for reproducibility.

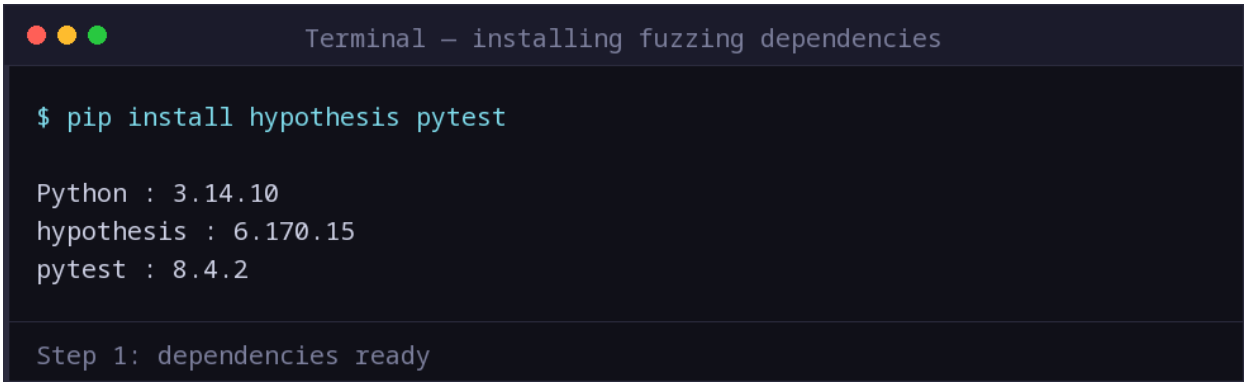


Figure 1 — Installing Hypothesis and pytest, and the versions in use.

3. Step 2 — Reproducing the buggy behaviour

Running the unmodified processor.py with well-formed input produces the expected results: a dimensional weight of 1.2 kg for a 20×30×10 cm parcel, the label E-000123 for the code e123, and a manifest weight of 12.5.

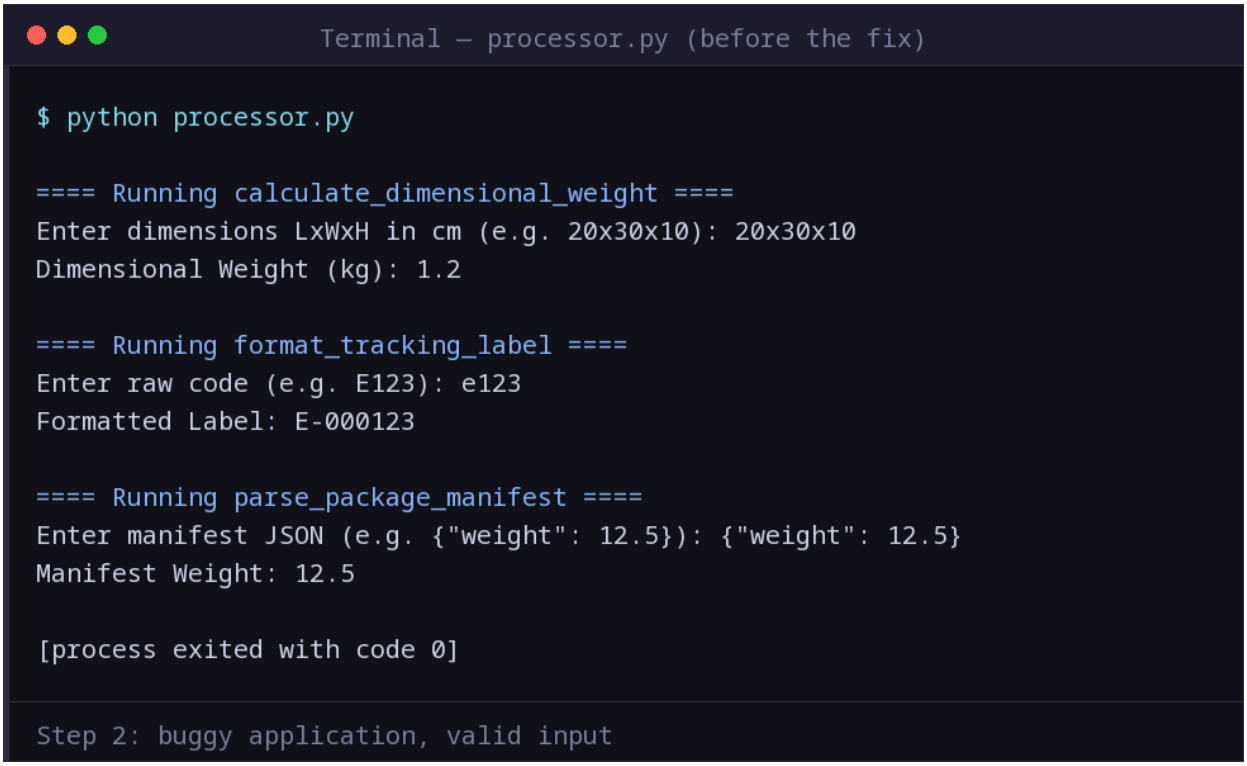


Figure 2 — processor.py on the happy path (valid input).

The same program is far less forgiving once the input is not perfect. Entering a truncated dimension string such as 30x reaches float("") and the process dies with a traceback instead of reporting a usable error. This is the class of defect the fuzzer is expected to find automatically.

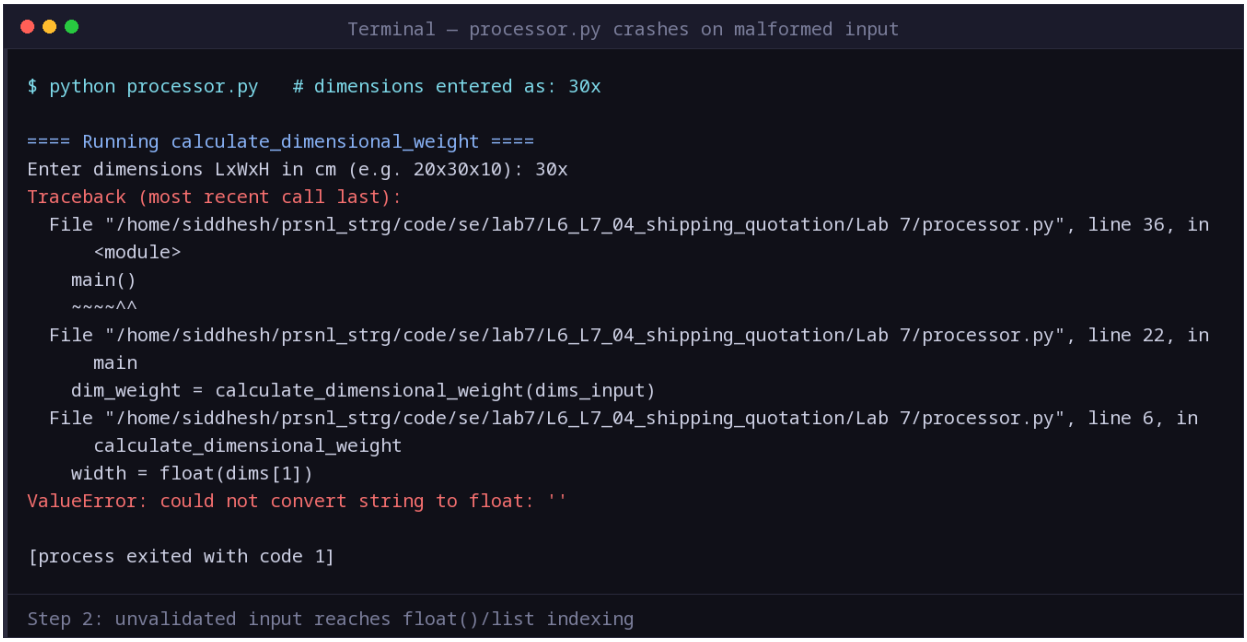


Figure 3 — malformed dimensions crash the application (ValueError traceback).

4. Step 3 — Completing the Hypothesis test suite

The three stubs in test_processor.py were completed so that every function is exercised with arbitrary text and None. A key design decision is how a hardened function is allowed to reject input: a validator must refuse bad input, so raising ValueError is treated as a valid outcome, while any other exception (IndexError, AttributeError, TypeError, JSONDecodeError, …) is a failure. A shared helper keeps that rule in one place and reports the input that triggered the crash.

```
def _call_no_crash(func, value):
    try:
        return func(value)
    except ValueError:
        return None                 # deliberate rejection: acceptable
    except Exception as exc:
        raise AssertionError(
            f"{func.__name__} crashed with {type(exc).__name__}: {exc} for input {value!r}"
        ) from exc


@given(st.text() | st.none())
def test_calculate_dimensional_weight_no_crash(s):
    result = _call_no_crash(calculate_dimensional_weight, s)
    if result is not None:
        assert isinstance(result, float)
        assert result >= 0
```

Beyond the three crash tests, four happy-path properties check that hardening does not change correct behaviour (the /5000 volumetric formula, the accepted separator spellings, label normalisation, and the JSON weight round-trip):

Test	Property asserted
test_calculate_dimensional_weight_no_crash	Arbitrary text or None never raises anything except ValueError; a returned value is a non-negative float.
test_format_tracking_label_safe	Arbitrary text or None never raises anything except ValueError; a returned label is <letter>-<digits>.
test_parse_package_manifest_safe	Arbitrary text or None never raises anything except ValueError; a returned weight is a non-negative float.
test_dimensional_weight_happy_path	For positive integer dimensions, round(L·W·H/5000, 2) is returned.
test_dimensional_weight_separator_variants	x, X, comma and ' * ' separators describe the same box and give the same weight.
test_tracking_label_happy_path	A lower-case prefix is normalised to PREFIX + six-digit zero-padded number.
test_manifest_weight_happy_path	A JSON manifest with a finite positive weight returns that weight.

5. Step 4 — Fuzzing results before the fix

Running pytest against the original code fails four of the seven tests within a fraction of a second. Hypothesis shrinks every failure to a minimal input, which makes the root cause immediately obvious (None, an empty string, or a separator that is not a lower-case x).

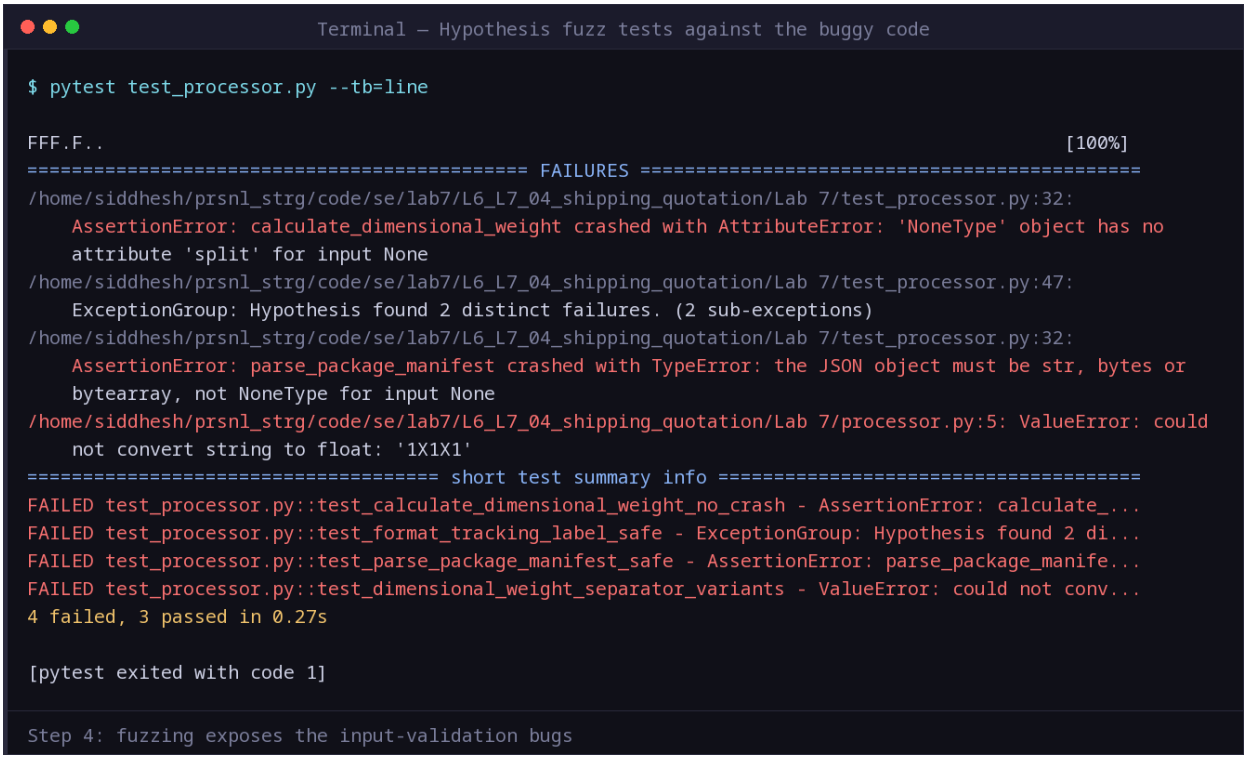


Figure 4 — pytest/Hypothesis output against the buggy processor.py (4 failed, 3 passed).

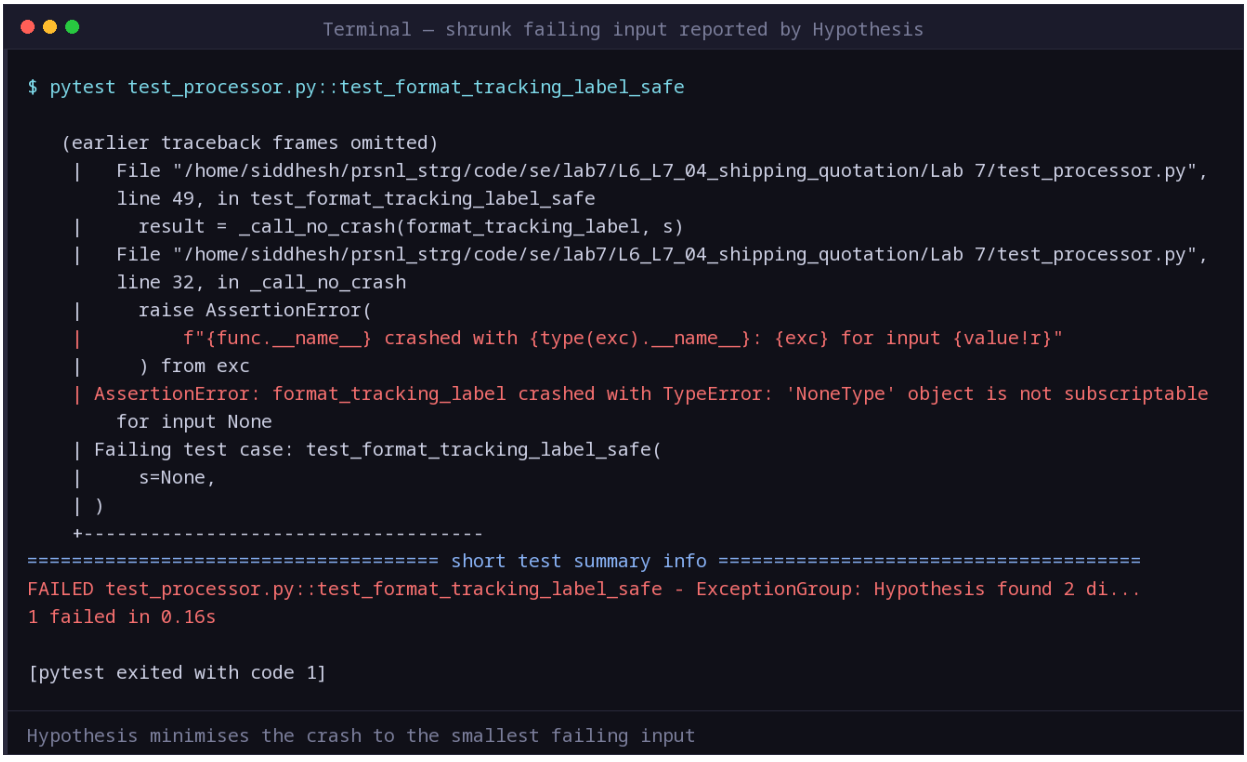


Figure 5 — the shrunk failing example for format_tracking_label: s=None.

The failures map onto nine distinct input-validation defects. In the table, DW = calculate_dimensional_weight, TL = format_tracking_label and PM = parse_package_manifest.

#	Func.	Failing input	Observed behaviour	Root cause
1	DW	None	AttributeError: 'NoneType' object has no attribute 'split'	Argument type is never checked.
2	DW	"30x", "1x2", ""	IndexError: list index out of range (or ValueError from float(""))	Splits and indexes dims[0..2] without counting the parts.
3	DW	"1X1X1", "20 x 30 x 10"	ValueError: could not convert string to float	Only lower-case 'x' is accepted; spaces are not stripped.

#	Func.	Failing input	Observed behaviour	Root cause
4	DW	"0x0x0", "-5x2x3", "nanx1x1"	Returns 0.0, a negative weight or nan	No positivity or finiteness checks on the dimensions.
5	TL	None, ""	TypeError: not subscriptable / IndexError: string index out of range	raw_code[0] is evaluated with no type or emptiness check.
6	TL	"E", "Eabc", "E-5", "123"	IndexError, ValueError, or a malformed label such as 1-000123	Assumes a letter followed by digits; negative numbers are reformatted.
7	PM	None, "", "{"	TypeError: JSON object must be str / JSONDecodeError	Payload type and JSON syntax are never validated.
8	PM	"{}", "[]", "null", {"weight": null}	KeyError: 'weight' / TypeError	JSON shape (object, required key, numeric value) is not checked.
9	PM	{"weight": "abc"}, {"weight": -2}, {"weight": NaN}	ValueError from float(), or NaN / negative weight accepted	Value is coerced without numeric, finite and positive checks.

Note that the fuzzer also found an inconsistency the manual tests missed: the upper-case separator "1X1X1" is rejected even though the label test shows the code intentionally normalises case. This is exactly the kind of asymmetric handling that property-based testing surfaces cheaply.

6. Step 5 — Hardening processor.py

All three functions now funnel invalid input through a single, documented rejection channel: ValueError with an explanatory message. The application entry point catches that exception and prints it, so the program degrades gracefully instead of terminating with a traceback.

Function	Hardening applied
calculate_dimensional_weight	Type check (must be str), strip whitespace, accept x / X / comma / * separators, require exactly three non-empty parts, parse each with float() inside try/except, reject non-finite and non-positive values, and guard against overflow in L·W·H.
format_tracking_label	Type check, strip whitespace, require a single alphabetic prefix, require a non-empty digit-only body (so E-5, Eabc and 123 are rejected), and format the parsed integer with a six-digit zero-padded field.
parse_package_manifest	Type check, catch JSONDecodeError with line/column, require a JSON object, require the weight key, require a numeric (non-boolean) value, and reject NaN, infinity, zero and negative weights.
main()	Each prompt is executed through a helper that catches ValueError and prints 'Invalid input: …', so one bad answer no longer aborts the remaining steps.

The fixed application still accepts the original happy-path input without any change in output:

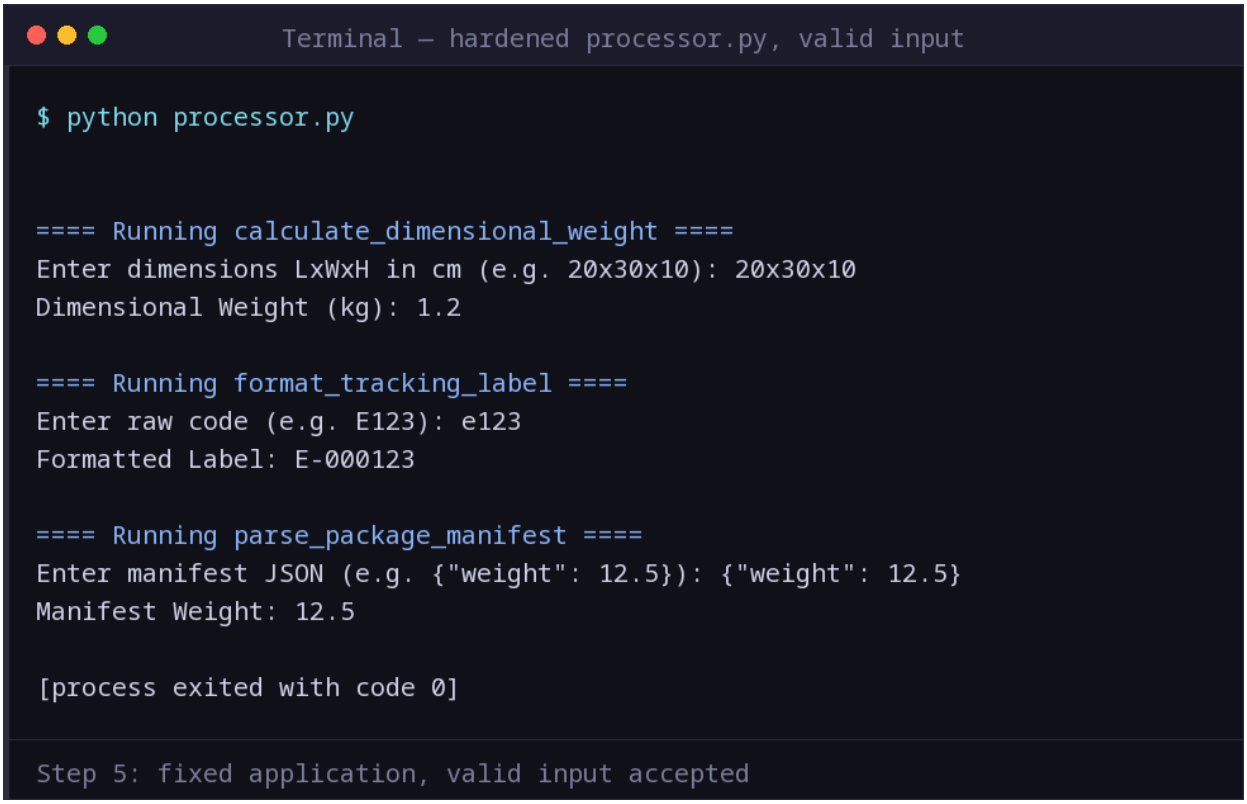


Figure 6 — hardened processor.py on valid input (identical results to Figure 2).

and it now reports malformed input as a message rather than a crash. All three prompts below are deliberately invalid (non-numeric dimensions, an empty tracking code, and a manifest without a weight):

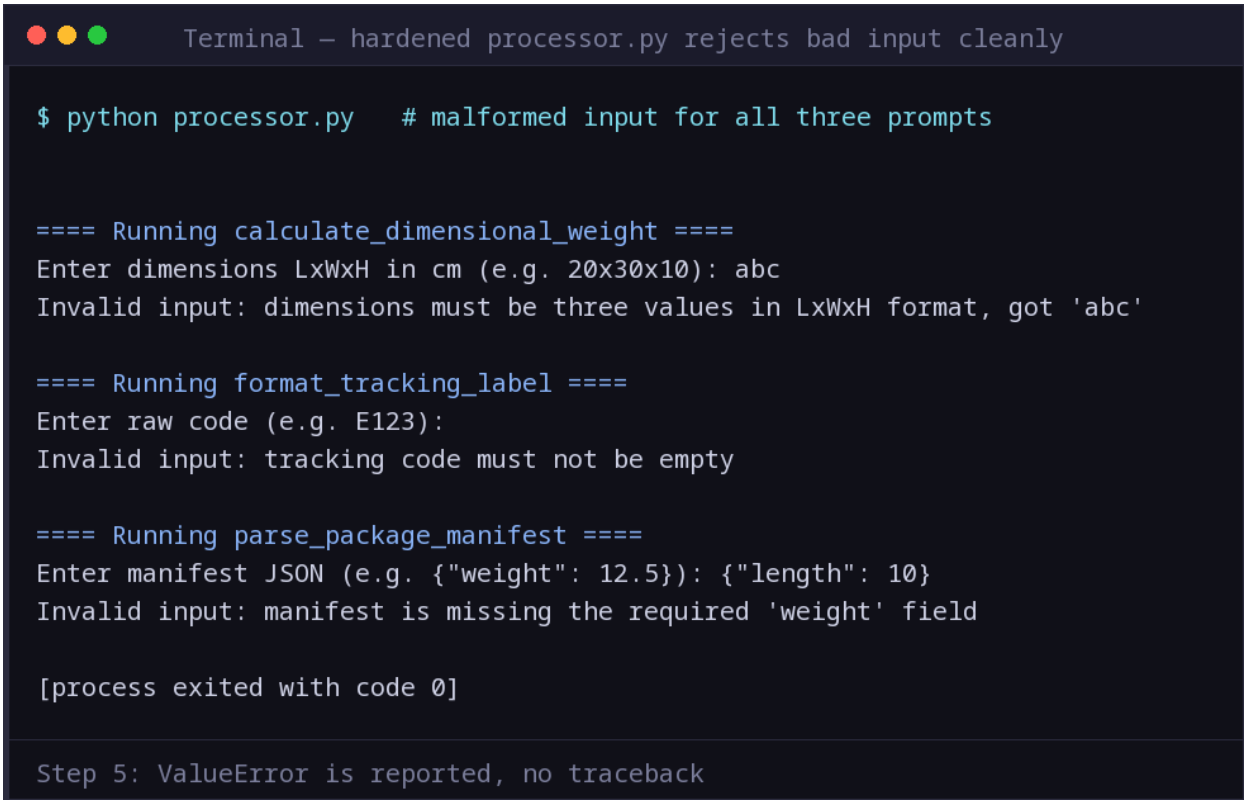


Figure 7 — the same three failures are now reported cleanly; the process exits 0.

7. Step 6 — Re-running the fuzz tests

The unchanged test suite is re-run against the hardened module. Every property holds: no unexpected exception type is ever raised on arbitrary text, and all happy-path results are preserved.

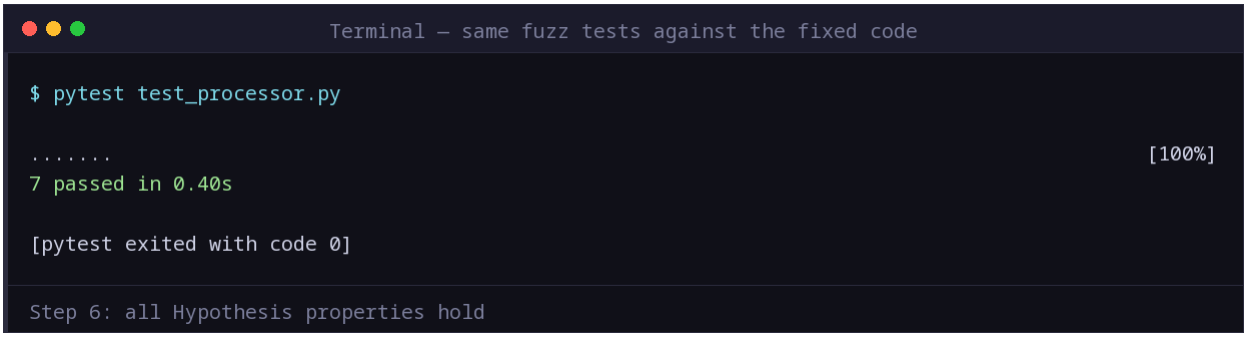


Figure 8 — the same seven Hypothesis tests pass against the fixed processor.py.

8. Discussion

- **Fuzzing complements unit tests.** Hypothesis needs no knowledge of specific bad values: describing the input type and a property ("never crashes", "never returns a negative number") is enough to discover None, empty strings, wrong separators and malformed JSON.
- **Minimisation is the real time-saver.** A shrunk failing example (s=None) reduces debugging to seconds compared with manually searching for an input that breaks the code.
- **Decide the rejection contract up front.** Accepting ValueError as "rejected input" keeps the crash property meaningful while allowing a validator to do its job; the alternative (never raise anything) would force the functions to return sentinel values and hide defects.
- **Validate once, at the edge.** Input validation belongs at the boundary. After the fix the rest of each function can rely on numeric, finite, positive values and on a JSON object with a real weight field.

9. Conclusion

Hypothesis exposed four failing properties in the original shipping-quotation processor and shrank them to minimal inputs in under a second of execution. The module was then hardened so that every malformed input is rejected with an explanatory ValueError, while valid input still produces the same results. The final suite (7 tests, hundreds of generated cases per run) passes, and the application no longer terminates with tracebacks when a user mistypes a value.

Appendix — Deliverables and reproduction

Deliverable	Location
Report (PDF)	Lab 7/Lab7_Fuzz_Testing_Report.pdf
Fixed application	Lab 7/processor.py
Completed fuzz tests	Lab 7/test_processor.py

```
cd "Lab 7"
pip install hypothesis pytest
python processor.py        # run the hardened application
pytest test_processor.py    # run the fuzz suite (7 passed)
```

The helper scripts, raw terminal captures, rendered screenshots and the DOCX draft of this report are kept in Lab 7/helper_files/ and are excluded from version control via .gitignore, along with the student handout PDF.